

תכנות מונחה עצמים

תרגול 5 : תבניות וטיפול בחריגות

מועד הגשה 11.06.2026 עד שעה 23:55

הוראות הגשה:

1. הגשה **באופן עצמאי** בלבד. הגשה בקבוצות תוביל לציון 0 בעבודה.
2. אין לשתף או להעתיק את העבודה או חלקים ממנה. עבירה על הוראה זו תוביל לציון 0 בעבודה.
3. **הגשה דרך מערכת מודל בלבד**. שום עבודה לא מתקבלת במייל!
4. התשובות לשאלות מחלק ג' צריכות לכלול 3 קבצים לכל שאלה: 2 קבצי ccpp (אחד למחלקה ואחד ל-main) וקובץ h. עבור חלק' ב' ו-ד' ניתן לייצר קובץ cpp אחד. את התשובות לחלק א' יש להגיש בפורמט טקסט. יש להגיש קובץ אחד בפורמט ZIP המכיל את כל קבצים של כל השאלות.
5. **אסור להוסיף משתנים למחלקה, ניתן להוסיף פונקציות (עזר) אבל עליהן להיות פרטיות.**

6. ניתן להשתמש בתרגיל בספריות iostream ו-string בלבד

7. שאלות ובקשות בקשר לעבודה להפנות אך ורק לאחראית התרגיל, ויקטוריה, במייל victoria@ac.sce.ac.i

חלק א' – תאורטי (מענה בקובץ טקסט – 5 נקודות)

1. מה יודפס בקוד הבא ולמה?

```
template<typename T>
void func(T t) {
    std::cout << "General\n";
}

template<>
void func<int>(int t) {
    std::cout << "Specialized\n";
}

int main() {
    func(10);
    func(3.14);
}
```

2. מה עושה קוד הבא?

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

ומה יקרה אם נקרא:

```
max(3, 4.5);
```

3. תסבירו, למה בתבנית מחלקה כל פונקציות צריכות להיות ממומשות בקובץ ה-h?

4. מה יידפס קוד? האם הקוד בכלל חוקי? נמקי תשובתך.

```
try {
    throw std::runtime_error("error");
}
catch(...) {
    std::cout << "catch all\n";
}
catch(const std::exception& e) {
    std::cout << "exception\n";
}
```

5. מה קורה במקרה הבא?

```
void f() {
    throw std::runtime_error("fail");
}
int main() {
    try {
        f();
    }
    catch(...) {
        std::cout << "caught";
    }
}
```

חלק ב' – תבנית לפונקציה

בחלק זה שתי משימות נפרדות.

משימה 1: תבנית של פונקציה (20 נקודות)

כתבו תבנית של פונקציה בוליאנית `isContain` המקבלת מערך כללי ומשתנה בסוג של המערך. התבנית לפונקציה תבדוק האם הערך (של משתנה) מופיע במערך הנתון. כתבו תכנית ראשית אשר תשתמש בפונקציה הנ"ל עבור הסוגים הבאים: `int, long, float, char, string`.

חלק ג' – תבנית למחלקה

משימה 2: תבנית של מחלקה (55 נקודות)

הגדרה: קבוצה הינה אוסף מוגדר היטב של איברים ייחודיים (ללא חשיבות לסדר כלשהו). אנו נייצג קבוצה על ידי שימוש במבנה נתונים מסוג **מערך דינאמי**. במשימה זו תבנו את המחלקות הבאות:

1. מחלקה של קבוצת איברים מסוג כללי

יש ליצור מחלקה בשם `CSet` המייצגת קבוצת ערכים מסוג כללי תוך שימוש בשדה שהוא מערך דינאמי וגודלו. יש ליצור עבור מחלקה:

- בנאי ברירת מחדל (יוצר קבוצה ריקה).
- בנאי מעתיק.
- הורס שמשחרר את הזיכרון של קבוצה.
- אופרטור השמה = המעדכן את האובייקט מסוג הנ"ל הנוכחי (LValue) ע"י אובייקט הפרמטר (RValue).

יש ליצור את המתודות ואופרטורים הבאים עבור המחלקה לטיפול בקבוצות:

1. אופרטור `+=` המקבל ערך חדש ומוסיף אותו לסוף הקבוצה רק במידה והערך אינו קיים בקבוצה, אחרת לא תתבצע אף פעולה נוספת (עדיף להדפיס הודעה). לדוגמה: נניח כי ישנה קבוצה מספרים שלמים מסוימת $A = (2, 4, 6, 7)$ ללא חשיבות לסדר כלשהו. בהינתן הפעלת אופרטור זה על קבוצת A עם מספר 8, הקבוצה חדשה תהינה: $(2, 4, 6, 7, 8)$.
2. אופרטור `-=`, המקבל ערך, במידה וערך מופיע בקבוצה, אזי הוא יוציא אותו מקבוצה זו, אחרת לא תתבצע אף פעולה נוספת (עדיף להדפיס הודעה).

3. פונקציה **Union** הפועלת על קבוצה ומקבלת כפרמטר קבוצה נוספת. תוצאה הפונקציה הינו קבוצה חדשה כאיחוד שתי הקבוצות (כלל האיברים הייחודיים משניהן).

לדוגמה: איחוד של (3, 1) ו-(18, 3), היא קבוצה (3,1,18).

4. אופרטור **פלט <** להדפסת הקבוצה באופן מסודר.
ניתן להוסיף מתודות (פונקציות המחלקה) עזר.

2. מחלקה תפריט

מחלקה תפריט **Menu** תנהל לנו את כלל הקבוצות.
היא תבנה 5 אובייקטים מהתבנית הנ"ל:

- קבוצה אחת של מספרים שלמים (int),
- קבוצה אחת של מספרים שלמים ארוכים (long),
- שתי קבוצות של תווים (char),
- קבוצה אחת של מילים (string).

1. הוספת איבר חדש לקבוצה:

לאחר בחירה זאת, על המשתמש לבחור לאיזה קבוצה הוא רוצה להוסיף איבר וכמו-כן להזין את האיבר החדש.

2. מחיקת איבר מהקבוצה הקיימת:

לאחר בחירה זאת, על המשתמש לבחור מאיזו קבוצה הוא רוצה להוציא את האיבר וכמו-כן עליו לבחור את האיבר שברצונו להוציא.

3. איחוד שתי קבוצות המכילות תווים:

לאחר בחירה זאת המערכת תדפיס על המסך את התוצאה של פעולת האיחוד בין קבוצות התווים.

4. הדפסת קבוצה על המסך:

לאחר בחירה זאת, על משתמש לבחור איזו קבוצה ברצונו להדפיס ואז להדפיס אותה.

5. יציאה:

לאחר בחירה זאת המערכת תצא בצורה מסודרת.

הערה חשובה: שימוש במחלקה בנויה או ב-STL יוביל לאיפוס ציון המשימה כולה
(כולל החלק של קבצים!)

חלק ד' – טיפול בחריגות (20 נקודות)

כתבו את הפונקציה `char* stringMultiply(const char *str, int n)` פעולת הפונקציה:

הפונקציה מקצה זיכרון דינמי וממלאת אותו ב-n שכפולים רציפים של המחרוזת str
(כולל תו סיום המחרוזת '\0').

הפונקציה מחזירה את הכתובת של הזיכרון שהוקצה.
לדוגמה:

עבור הקריאה `stringMultiply("abcd", 4)` הפונקציה תחזיר מצביע למחרוזת:
"abcdabcdabcdabcd"

טיפול בחריגות (Exceptions):

במקרה של קלט לא תקין, הפונקציה תזרוק חריגה מסוג `std::invalid_argument`
הנושאת את הודעת השגיאה המתאימה:

מחרוזת ריקה: אם אורך המחרוזת הוא 0, הפונקציה תזרוק חריגה עם ההודעה:
"Empty string". לדוגמה,

`"Empty string" <- stringMultiply("", 4)`

ערך n לא חוקי: אם n הוא אפס או שלילי, הפונקציה תזרוק חריגה עם ההודעה:
"Invalid value :". בתוספת הערך המספרי של n. לדוגמה,

`"Invalid value : -3" <- stringMultiply("Hello", -3)`

**יש לכתוב תכנית ראשית main, המפעילה את הפונקציה עבור קלט
תקין ועבור מקרי קצה ומדפיסה על המסך את התוצאות ואת ההודעות
החריגות בהתאם.**